FACULDADE DE
CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA

Data Races

lecture 24 (2025-06-02)

Master in Computer Science and Engineering

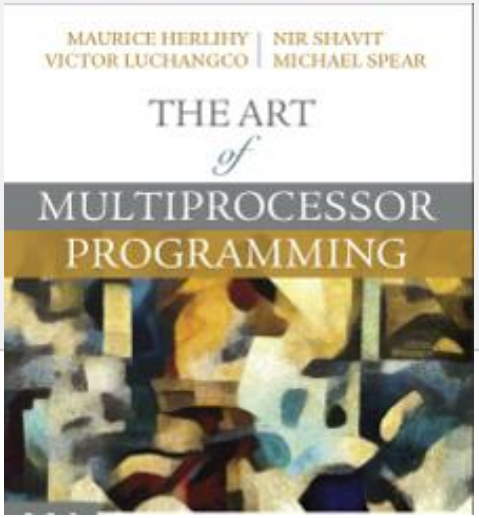
— Concurrency and Parallelism / 2024-25 —

João Lourenço <joao.lourenco@fct.unl.pt>

Based in the companion slides from “The Art of Multiprocessor Programming”

Outline

- (Low-Level) Data-Races
- High-Level Data Races
- Stale Values (Privatization)



Bibliography:

- Leslie Lamport. 1978.
  Time, clocks, and the ordering of events in a distributed system.
  Commun. ACM 21, 7 (July 1978), 558–565. DOI: 10.1145/359545.359563
- S. Savage, M. Burrows, G. Nelson, P. Sobalvarro, and T. Anderson. 1997.
  Eraser: a dynamic data race detector for multithreaded programs.
  ACM Trans. Comput. Syst. 15, 4 (Nov. 1997), 391–411. DOI: 10.1145/265924.265927

Concurrency Errors

Dynamic Data Race Detection Using Happens-before [Lamport '78]

Lock Definition

- **Lock**: a synchronization object that is either available, or owned (by a thread)
  - Operations: **lock(mu)** and **unlock(mu)**
    - (*We are assuming no explicit initialize operation*)
  - A lock can only be unlocked by its current owner
  - The **lock()** operation is blocking if the lock is owned by another thread

The Happens-before Relation

- *happens-before* (→) defines a partial order for events in a set of concurrent threads
  - In a single thread, *happens-before* reflects the temporal order of event occurrence
  - Between threads, **A** → **B** if **A** is an *unlock* access in one thread, and **B** is a *lock* access in a **different** thread
    - Assuming the threads obey the sequential specification of the lock , i.e.,

      can't have two successive locks, or two successive unlocks, or a lock in one thread and an unlock in a different thread

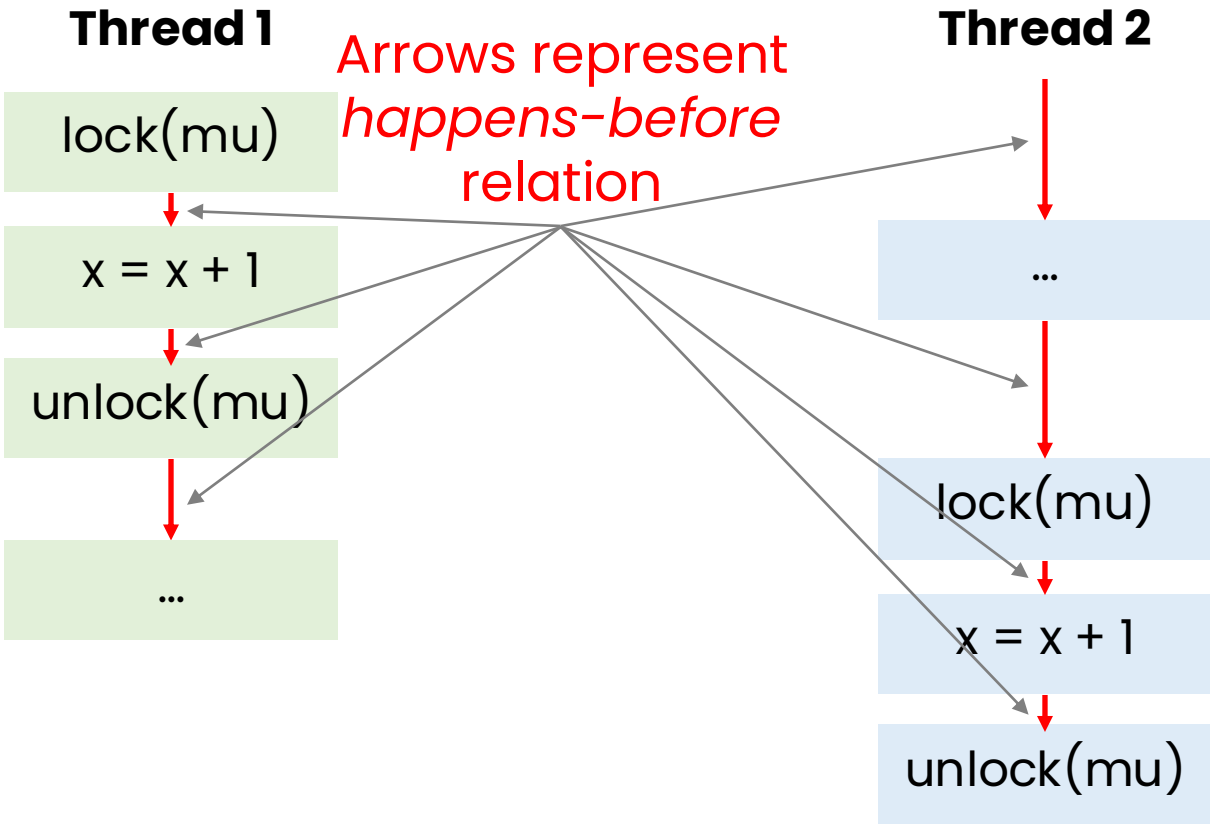
The Happens-before Relation

- Let **event a** be in thread 1 and **event b** be in thread 2

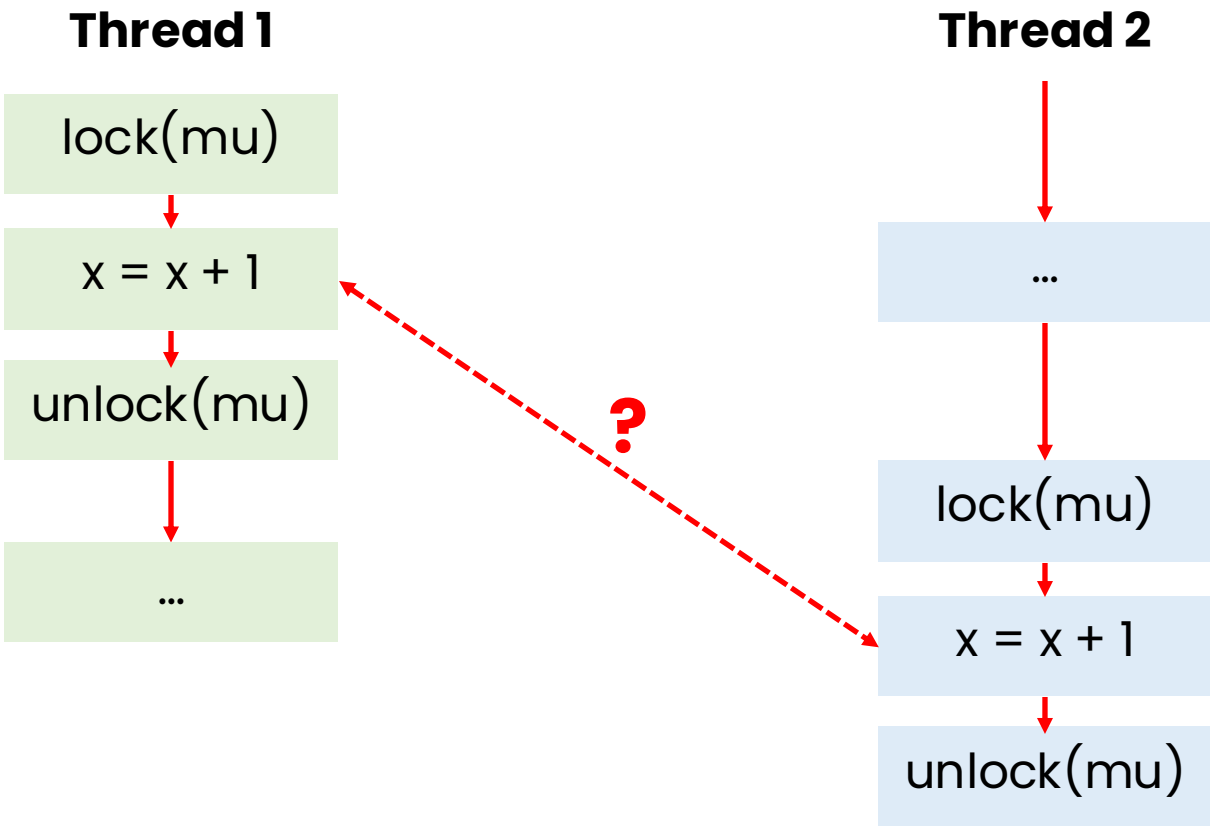
If $a = unlock(mu)$ ***and*** $b = lock(mu)$
then $a \rightarrow b$ (*a happens-before b*)

Data races between threads are ***possible*** if accesses to shared variables are not ordered by the *happens-before* relation

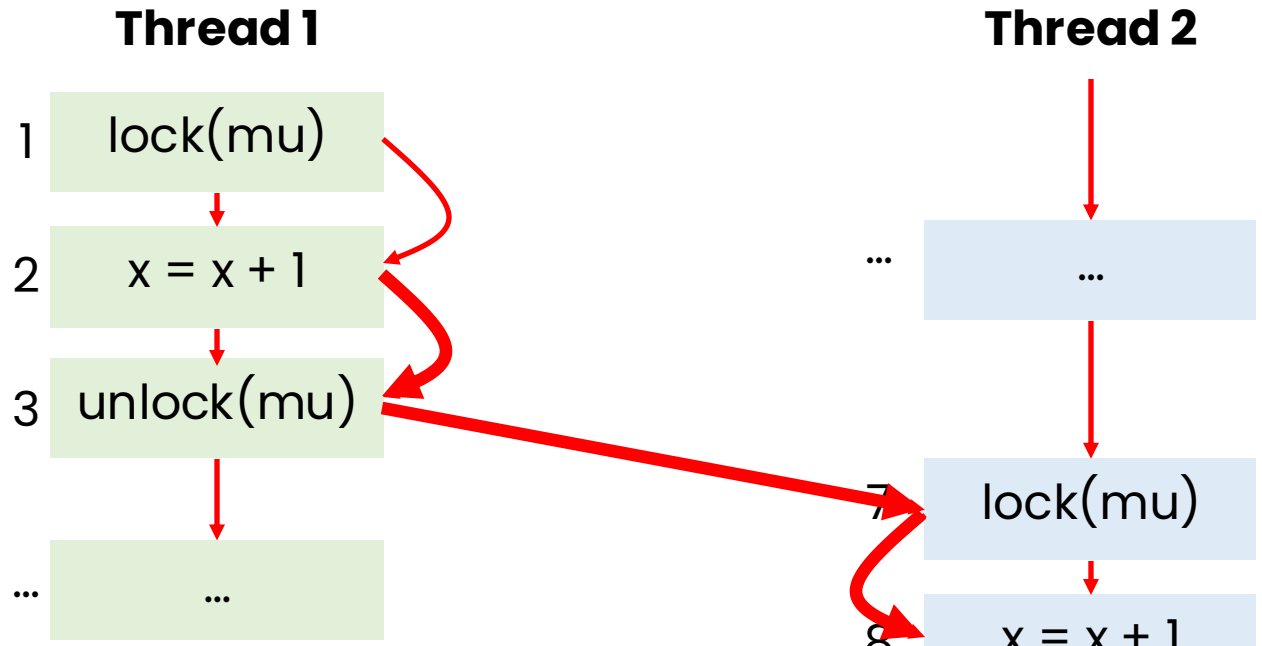
Example 1


Thread 1
Thread 2
Arrows represent
happens-before
relation
lock(mu)
x = x + 1
unlock(mu)
...
...
lock(mu)
x = x + 1
unlock(mu)

Example 1



Example 1



ev1 → *ev2* → *ev3*

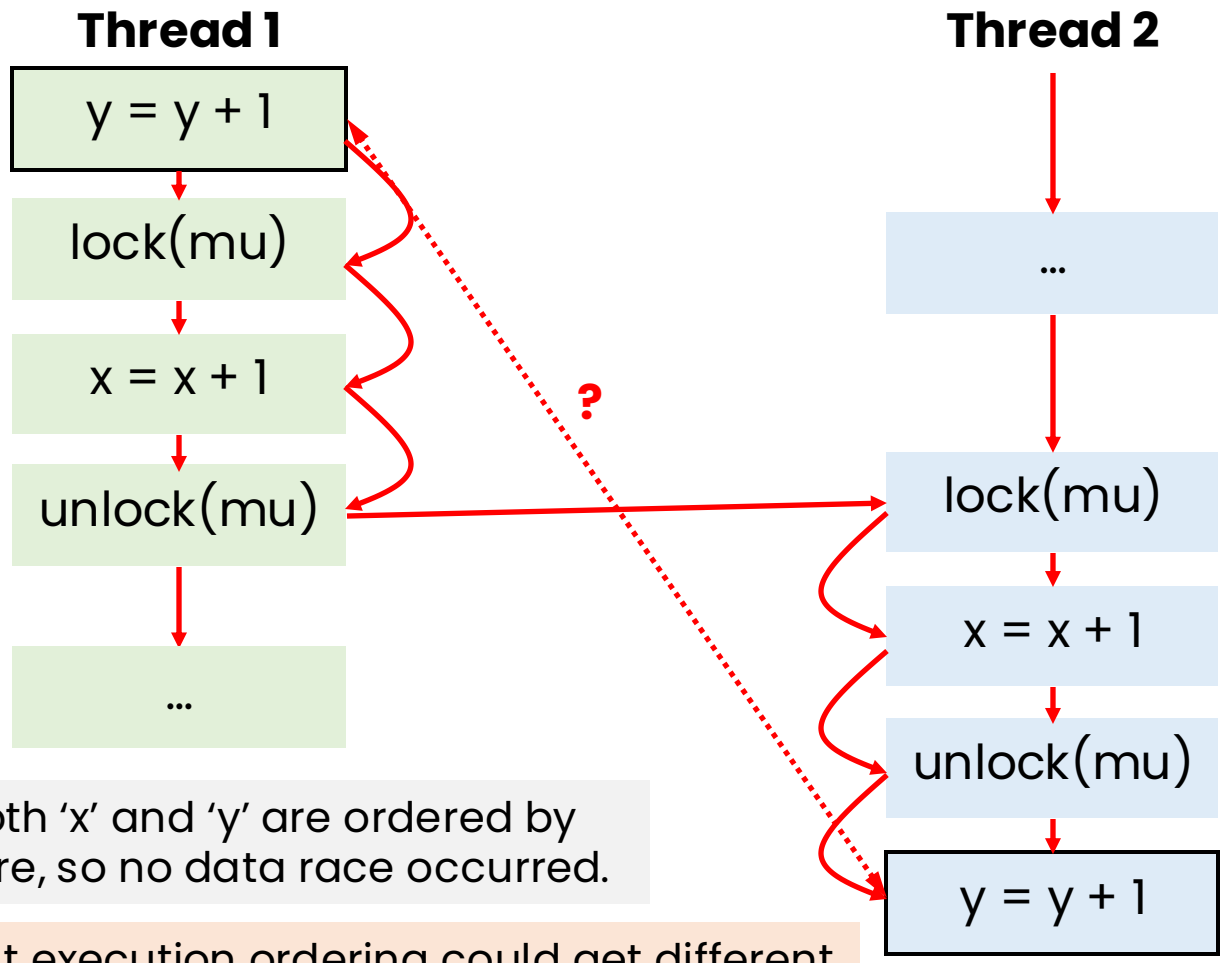
ev7 → *ev8* → *ev9*

ev3 → *ev7*

ev2* → *ev8

No Data Race

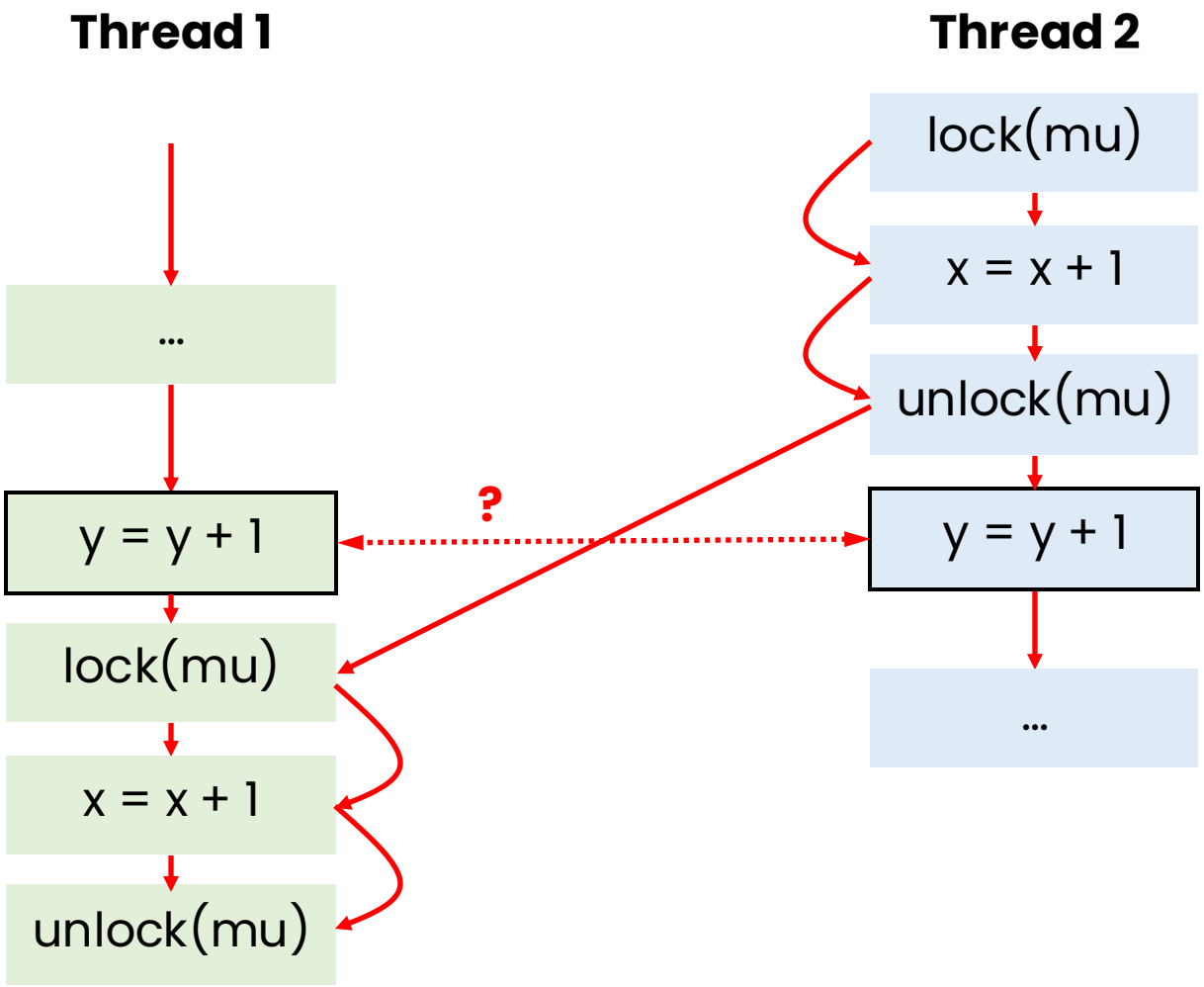
Example 2



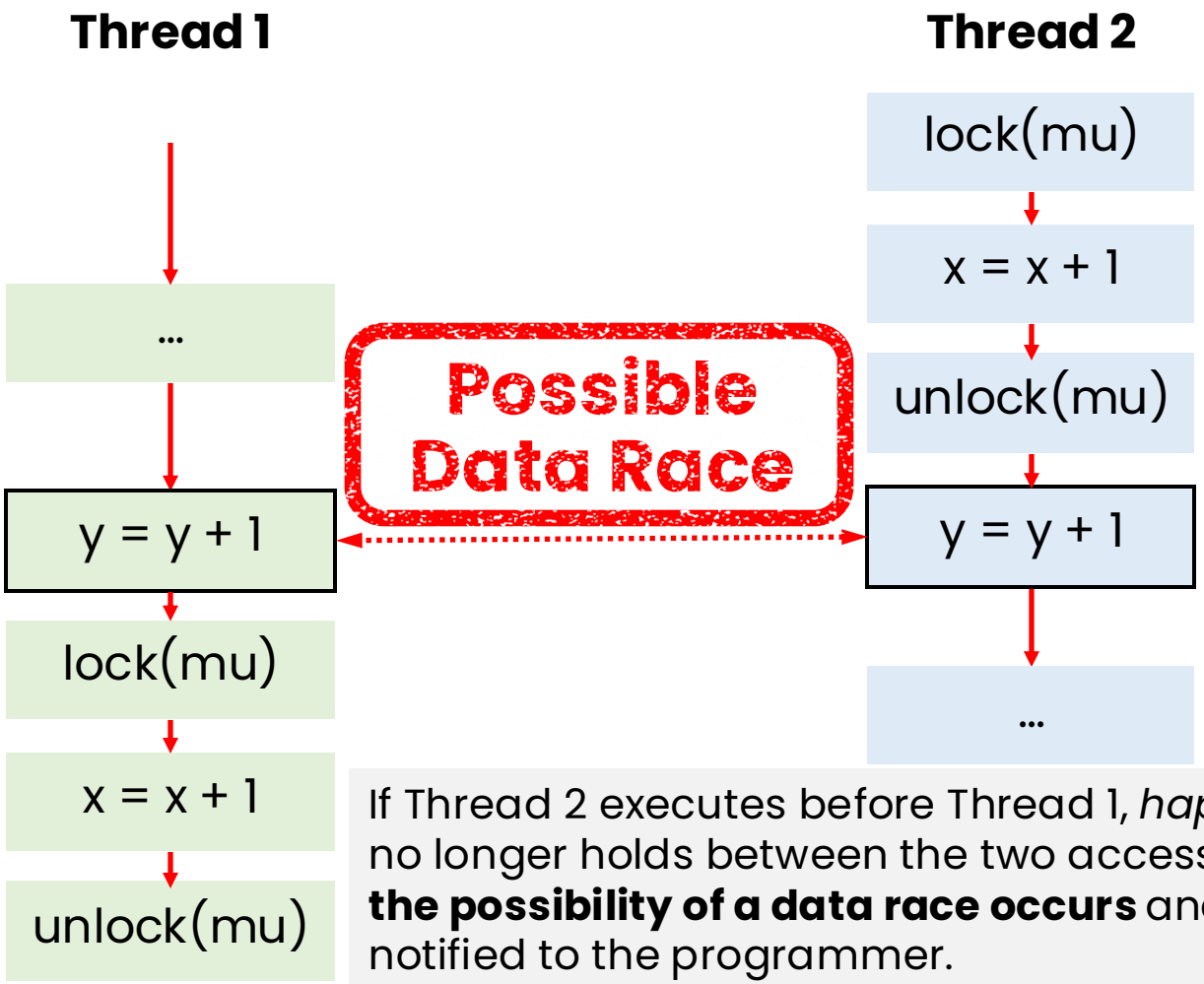
Accesses to both 'x' and 'y' are ordered by happens-before, so no data race occurred.

But … a different execution ordering could get different results?! Hppens-before only detects data races if the incorrect order shows up in the execution trace.

Example 3


Thread 1
Thread 2
lock(mu)
x = x + 1
unlock(mu)
...
y = y + 1
?
y = y + 1
lock(mu)
...
x = x + 1
unlock(mu)

Example 3



If Thread 2 executes before Thread 1, *happens-before* no longer holds between the two accesses to 'y', so **the possibility of a data race occurs** and should be notified to the programmer.

Concurrency Errors

The Lock-Set Algorithm — Eraser [Savage et.al. '97]

Approaches

- Check a sufficient condition for data-race freedom
- Consistent locking discipline
  - Every data structure is protected by a single lock
  - All accesses to the data structure are made while holding the lock

Thread 1

```
void Bank::Deposit(int a) {

    int t = bal;
    bal = t + a;

}
```

Thread 2

```
void Bank::Withdraw(int a) {

    int t = bal;
    bal = t - a;

}
```

Approaches

- Check a sufficient condition for data-race freedom
- Consistent locking discipline
  - Every data structure is protected by a single lock
  - All accesses to the data structure are made while holding the loc

Accesses to 'bal' are consistently protected by 'balLock'.

Thread 1

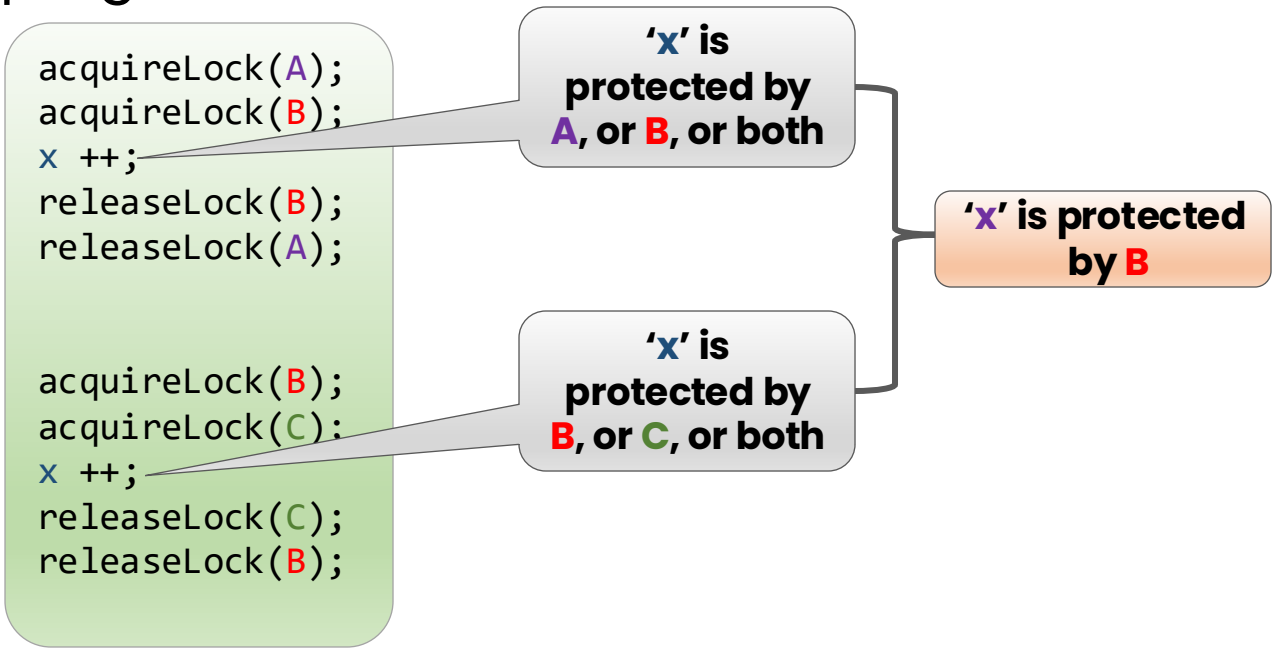
```
void Bank::Deposit(in
    acquireLock(balLock);
    int t = bal;
    bal = t + a;
    releaseLock(balLock);
}
```

Thread 2

```
Withdraw(int a) {
    acquireLock(balLock);
    int t = bal;
    bal = t - a;
    releaseLock(balLock);
}
```

Approach

- How to know which locks protect each memory location?
  - Ask the programmer? Cumbersome!
  - Infer from the program code? Is it effective?



The Lock-Set Algorithm

- Two data structures:
  - `LocksHeld(t)` = set of locks held currently by thread t
    - Initially set to Empty
  - `LockSet(x)` = set of locks that could potentially be protecting x
    - Initially set to the universal set
- When thread ‘t’ acquires lock ‘l’
  - `LocksHeld(t) = LocksHeld(t) ∪ {l}`
- When thread ‘t’ releases lock ‘l’
  - `LocksHeld(t) = LocksHeld(t) \ {l}`
- When thread ‘t’ accesses location ‘x’
  - `LockSet(x) = LockSet(x) ∩ LocksHeld(t)`
- “Data race” warning if `LockSet(x)` becomes empty

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)		
lock(m2)		
v = v + 1		
unlock(m2)		
v = v + 2		
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	**{m1}**	{m1, m2}
lock(m2)		
v = v + 1		
unlock(m2)		
v = v + 2		
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	**{m1, m2}**	{m1, m2}
v = v + 1		
unlock(m2)		
v = v + 2		
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2} ∩	**{m1, m2}**
unlock(m2)		
v = v + 2		
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

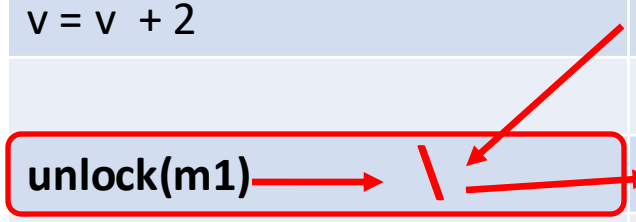
Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	**{m1}**	{m1, m2}
v = v + 2		
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1} ∩	**{m1}**
unlock(m1)		
lock(m2)		
v = v + 1		
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1}	{m1}
unlock(m1)	**{ }**	{m1}
lock(m2)		
v = v + 1		
unlock(m2)		



Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1}	{m1}
unlock(m1)	{ }	{m1}
lock(m2) → U	**{m2}**	{m1}
v = v + 1		
unlock(m2)		

Another Example

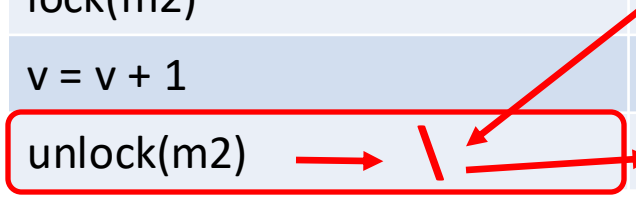
Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1}	{m1}
unlock(m1)	{ }	{m1}
lock(m2)	{m2}	{m1}
v = v + 1	{m2} ∩	**{ }**
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1}	{m1}
unlock(m1)	{ }	{m1}
lock(m2)	{m2}	{m1}
v = v + 1		**{ } — ALARM**
unlock(m2)		

Another Example

Program Code	LocksHeld	LockSet
	{ }	{m1, m2}
lock (m1)	{m1}	{m1, m2}
lock(m2)	{m1, m2}	{m1, m2}
v = v + 1	{m1, m2}	{m1, m2}
unlock(m2)	{m1}	{m1, m2}
v = v + 2	{m1}	{m1}
unlock(m1)	{ }	{m1}
lock(m2)	{m2}	{m1}
v = v + 1		**{ } — ALARM**
unlock(m2)	{ }	{ }



Algorithm Guarantees

- No warnings => no data races on the current execution
  - I.e., *the program followed consistent locking discipline in this execution*
- Warnings does not imply a data race
  - Just a *bad locking discipline*

Thread 1

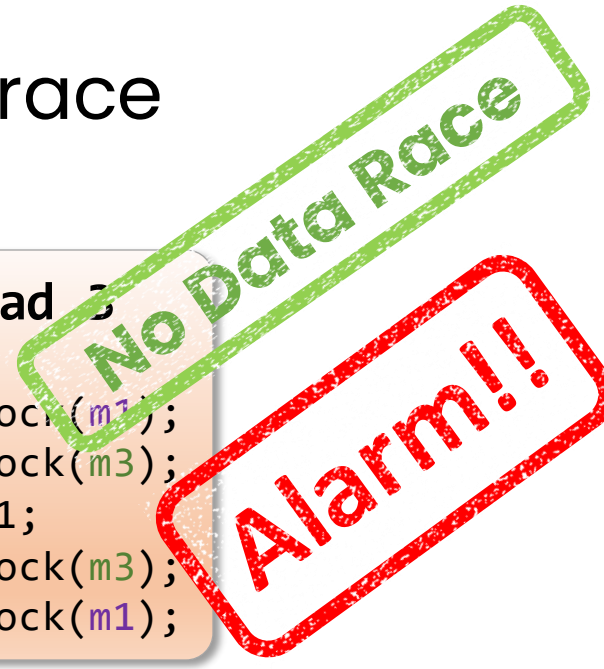
```
acquireLock(m1);
acquireLock(m2);
x = x + 1;
releaseLock(m2);
releaseLock(m1);
```

Thread 2

```
acquireLock(m2);
acquireLock(m3);
x = x + 1;
releaseLock(m3);
releaseLock(m2);
```

Thread 3

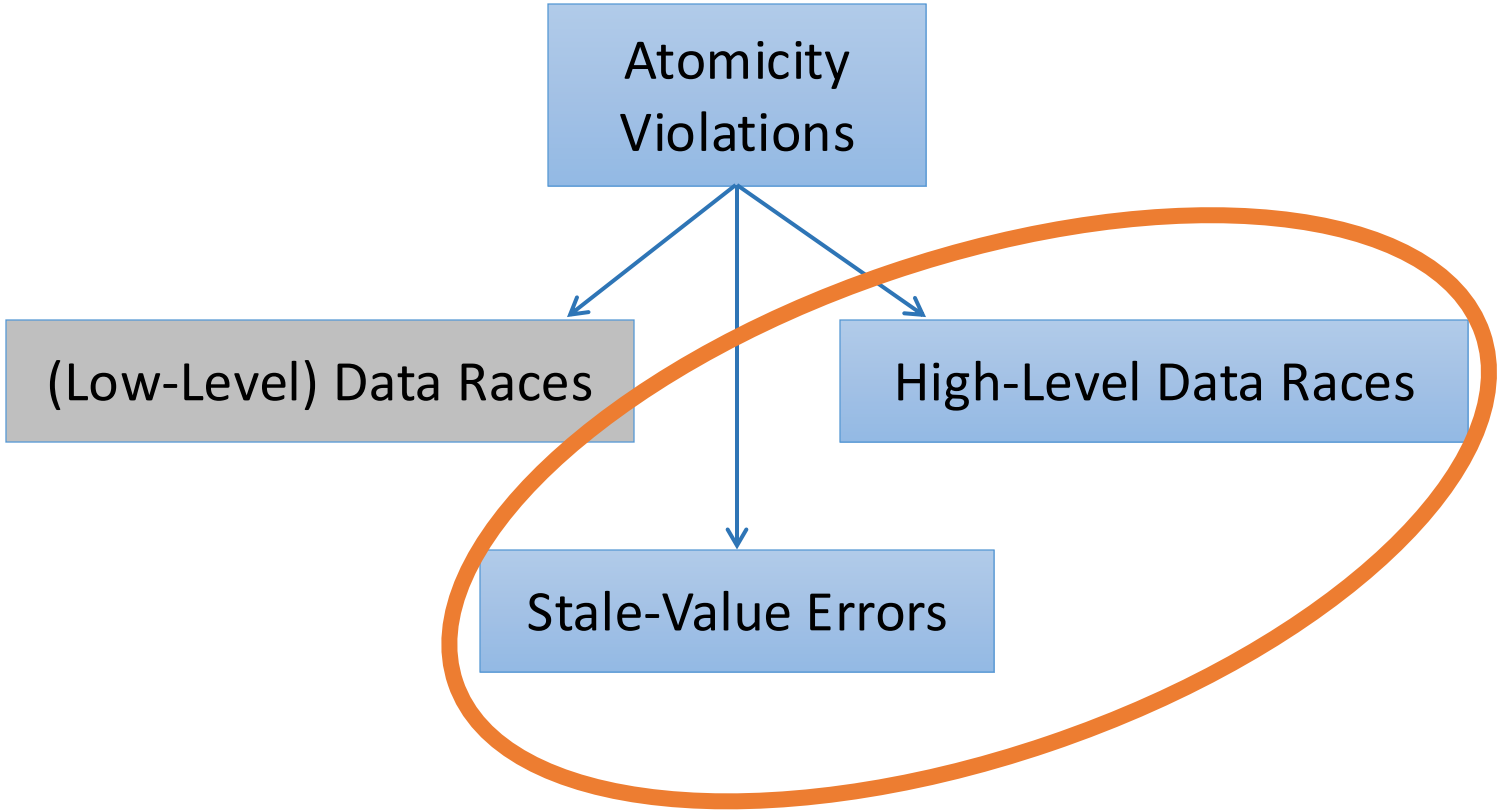
```
acquireLock(m1);
acquireLock(m3);
x = x + 1;
releaseLock(m3);
releaseLock(m1);
```



Concurrency Errors

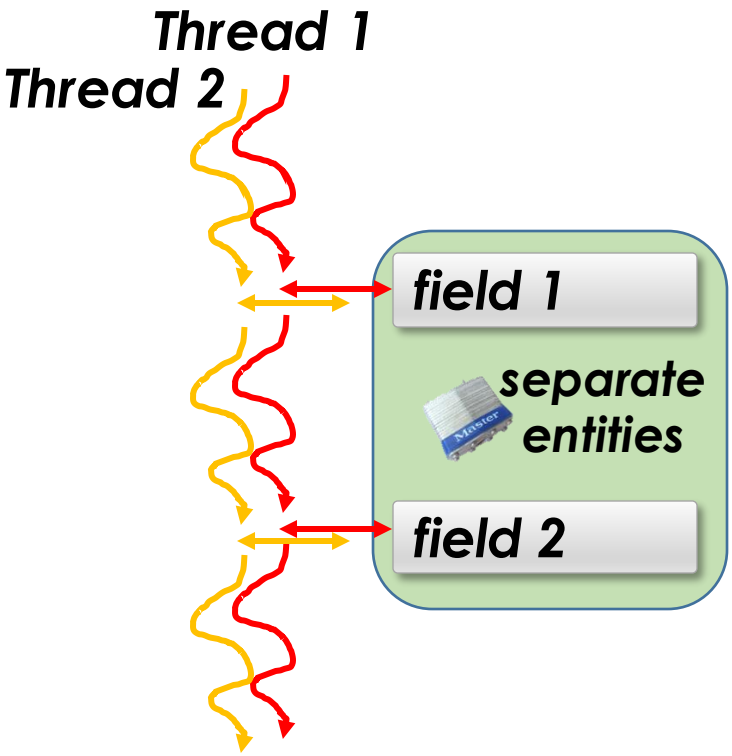
Detection of High-level Data Races and Stale-value Errors [Artho03]

Concurrency Anomalies



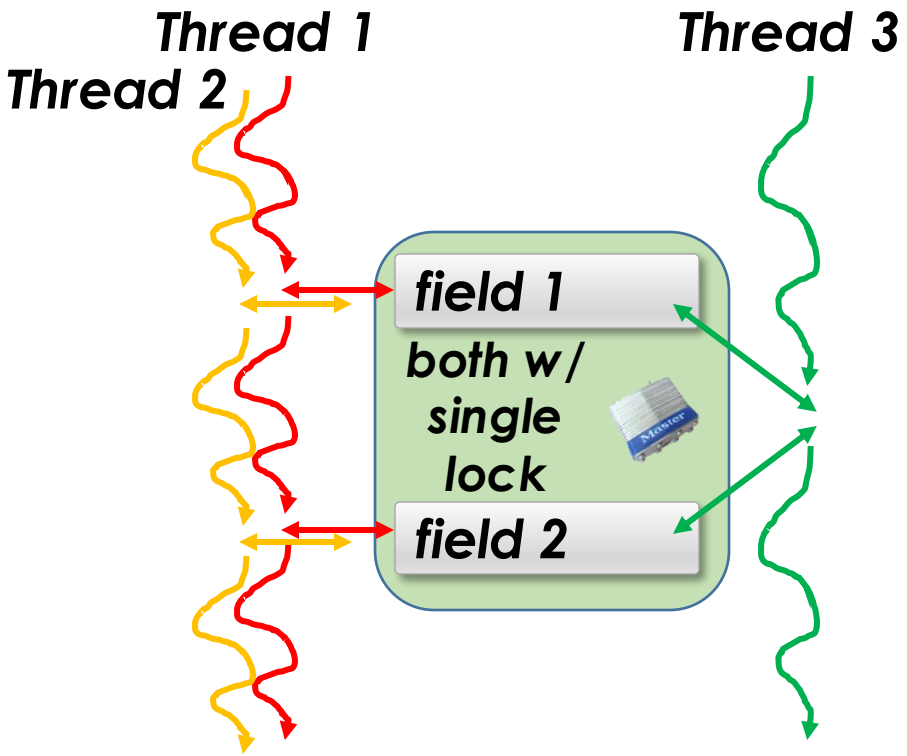
High-Level Data Race

- Wrongly defined atomic blocks



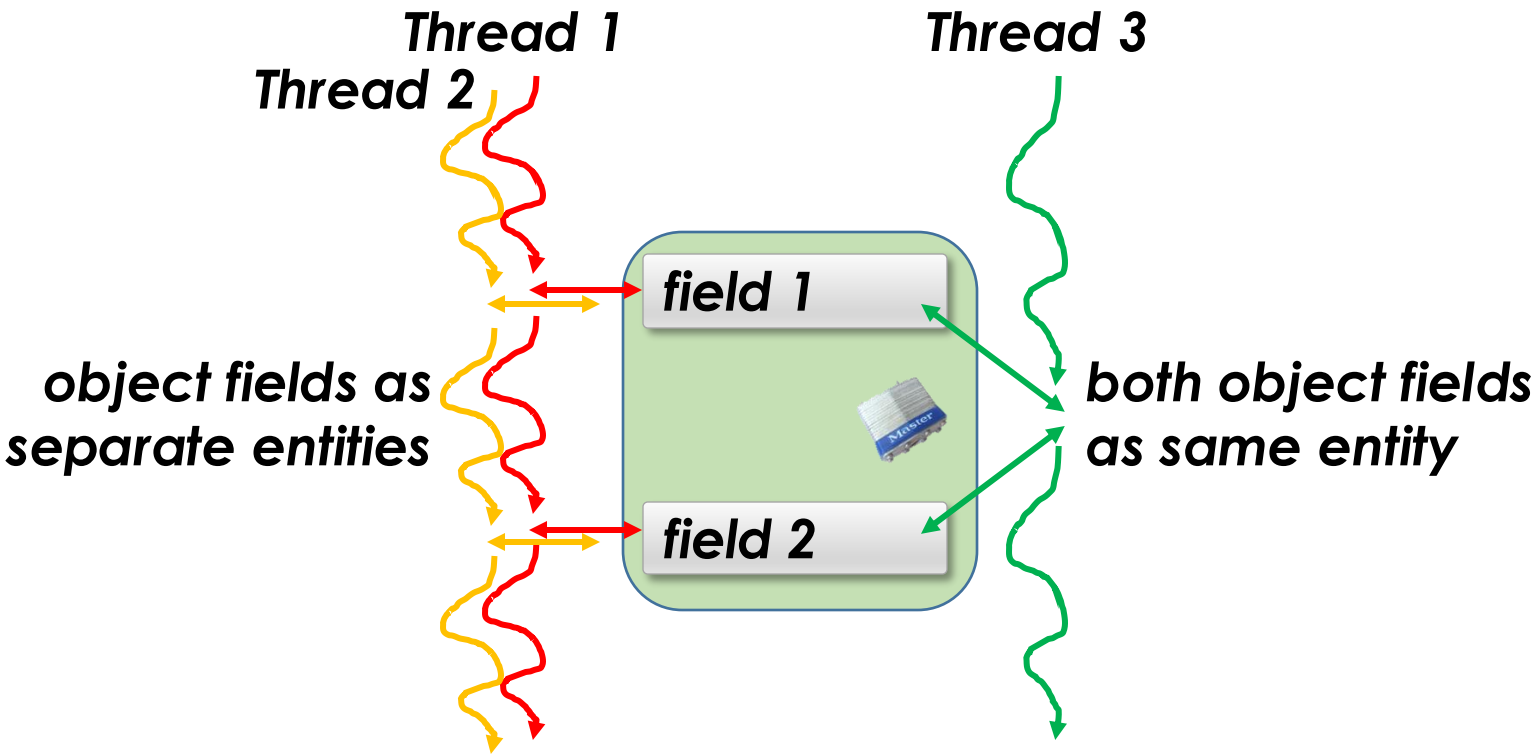
High-Level Data Race

- Wrongly defined atomic blocks



High-Level Data Race

- Wrongly defined atomic blocks



High-Level Data Races

```
public synchronized
int getX(){
  return this.x;
}
```

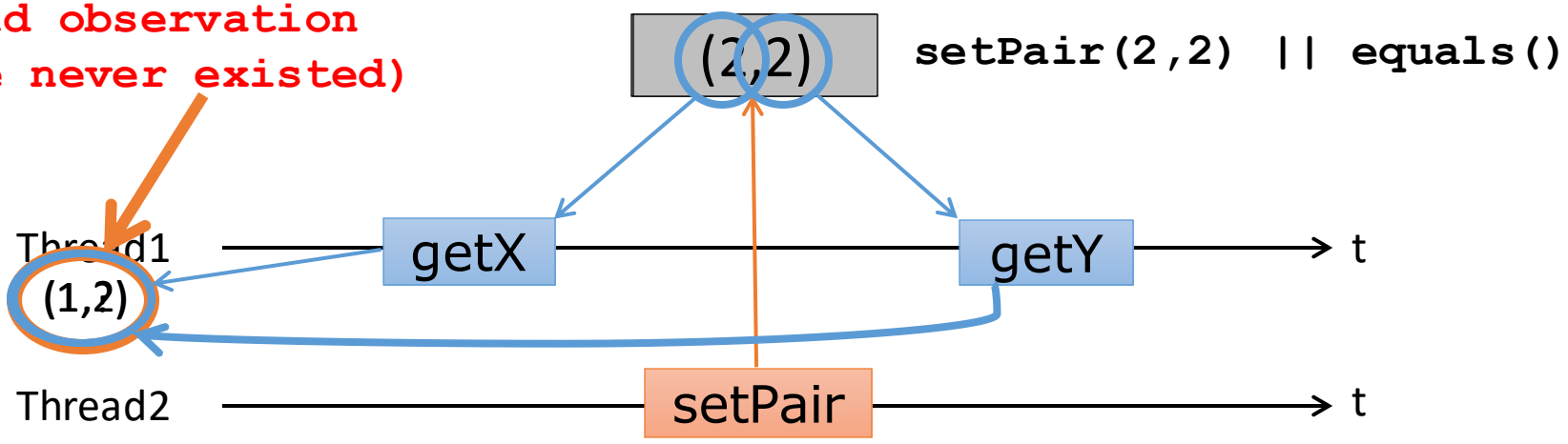
```
public synchronized
int getY(){
  return this.y;
}
```

Shared variables: x, y

```
// Thread 1
public boolean equals(){
  int loc_x = getX(); // synchr
  int loc_y = getY(); // synchr
  return loc_x == loc_y;
}
```

```
// Thread 2
public synchronized
int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

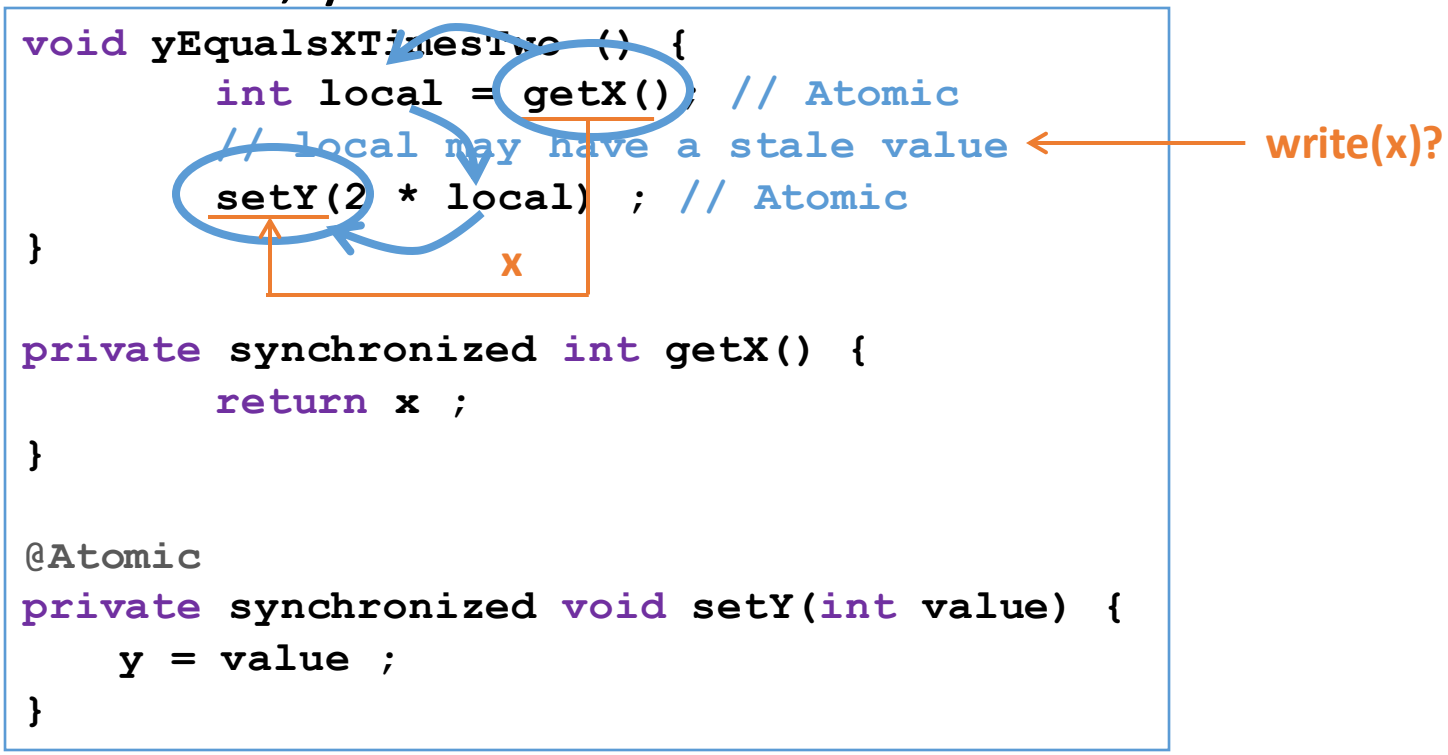
Invalid observation
(state never existed)



Stale-Value Errors

- Caused by privatization of shared values

  Shared variables: x, y



View of an Atomic Block [Artho03]

- A view of an atomic block B — V(B) — is the set of variables accessed inside the atomic code block B

View of an Atomic Block

- A view of an atomic block B — V(B) — is the set of variables accessed inside the atomic code block B
- The *read view* of B — $V_R(B) \subseteq V(B)$ — is the set of variables read inside the atomic code block B
- The *write view* of B — $V_W(B) \subseteq V(B)$ — is the set of variables written inside the atomic code block B

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

V(incX) = ?

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

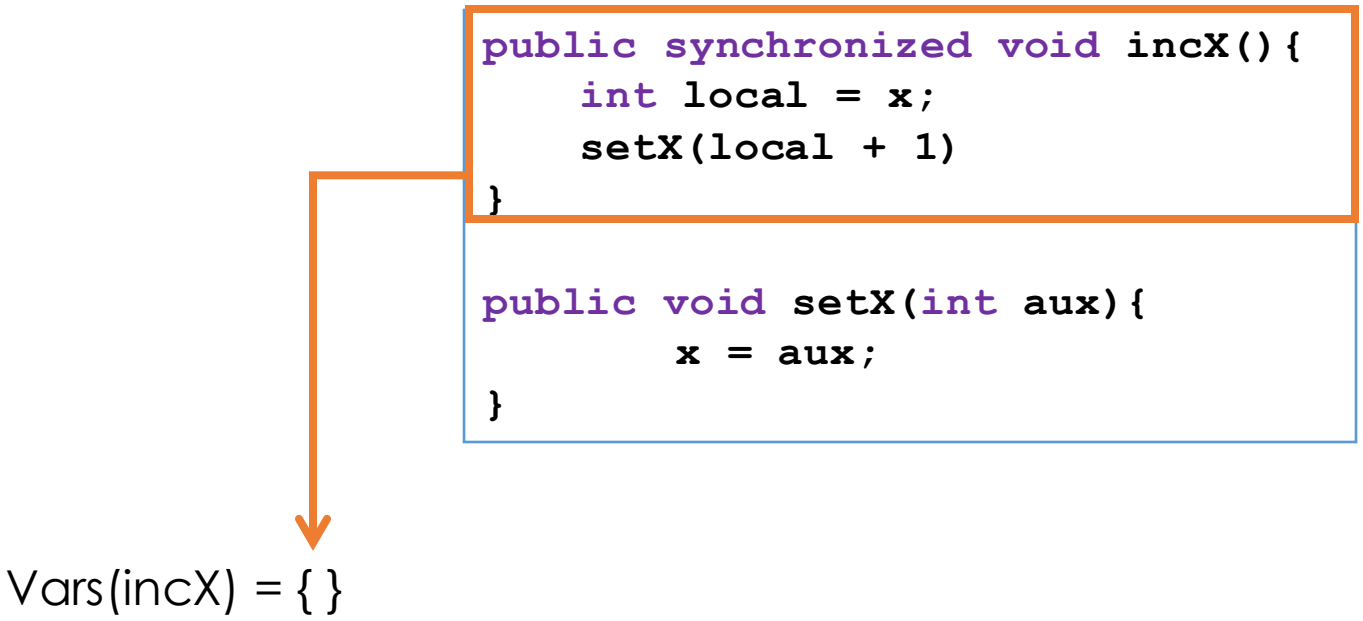
```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Which (shared) variables are accessed and how?

Views Analysis [Artho03]

- View: set of shared variables accessed atomically



Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)}

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)} ∪ Vars(setX)

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```


Vars(incX) = { } ∪ {read(X)} ∪ Vars(setX)

Vars(setX) = { }

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)} ∪ Vars(setX)

Vars(setX) = { } ∪ {write(X)}

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)} ∪ Vars(setX)

Vars(setX) = {write(X)}

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)} ∪ Vars(setX)

Vars(setX) = {write(X)}

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = { } ∪ {read(X)} ∪ {write(X)}

Views Analysis [Artho03]

- View: set of shared variables accessed atomically

```
public synchronized void incX(){
    int local = x;
    setX(local + 1)
}

public void setX(int aux){
        x = aux;
}
```

Vars(incX) = {read(X), write(X)} **V(incX) = { X }**

Views Analysis

- View: set of shared variables accessed atomically

```
// Thread 1
public boolean equals(){
  int loc_x = getX(); // Atomic
  int loc_y = getY(); // Atomic
  return loc_x == loc_y;
}
```

```
// Thread 2
public synchronized
int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

```
public synchronized int getX(){
return this.x;
}
```

```
public synchronized int getY(){
return this.y;
}
```

Views Analysis

```
public int getX(){
return this.x;
}
```

V(getX) = { X }

```
public int getY(){
return this.y;
}
```

V(getY) = { Y }

```
public int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

V(setPair) = { X, Y }

```
public boolean equals(){
  int loc_x = getX();
  int loc_y = getY();
  return loc_x == loc_y;
}
```

V(equals) = { X, Y }

High-Level Data Race

Thread 1

```
public synchronized
int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

Thread 2

```
public boolean equals(){
  int loc_x = getX(); // Atomic
  int loc_y = getY(); // Atomic
  return loc_x == loc_y;
}
```

High-Level Data Race

Thread 1

```
public synchronized
int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

Thread 2

```
public boolean equals(){
  int loc_x = getX()   // Atomic
  int loc_y = getY()   // Atomic
  return loc_x == loc_y;
}
```

Thread 1	Thread 2
V (setPair) = {x, y}	V (getX) = {x}
	V (getY) = {y}

High-Level Data Race

Thread 1

```
public synchronized
int setPair(int v1, int v2){
    x = v1;
    y = v2;
}
```

Thread 2

```
public boolean equals(){
  int loc_x = getX(); // Atomic
  int loc_y = getY(); // Atomic
  return loc_x == loc_y;
}
```

Maximal View of T1

Thread 1	Thread 2
V (setPair) = {x, y}	V (getX) = {x} — View of T2
	V (getY) = {y} — View of T2

$\{x, y\} \cap \{x\} = \{x\}$

$\{x, y\} \cap \{y\} = \{y\}$

$(\{x\} \not\subseteq \{y\} \wedge \{y\} \not\subseteq \{x\})$

Data Race

HLDR Quiz (1)

- T1 runs **V1 = {A, B, C}** and **V2 = {A, B, C, D}**
- T2 runs **V3 = {A, B, D, E}** and **V4 = {A, D, E}**
- **Is there a HLDR?**

- V2 is maximal in T1 (ignore V1)

- V2 ∩ V3 = {A, B, D} V2 ∩ V4 = {A, D}
- {A, B, D} ⊆ {A, D} or {A, D} ⊆ {A, B, D}? **Yes!**

HLDR Quiz (2)

- T1 runs **V1 = {A, B, C}** and **V2 = {A, B, C, D}**
- T2 runs **V3 = {A, B, E}** and **V4 = {B, C, F}**
- **Is there a HLDR?**

- V2 is maximal in T1 (ignore V1)

- $V2 \cap V3 = \{A, B\}$ $V2 \cap V4 = \{B, C\}$
- $\{A, B\} \subseteq \{B, C\}$ or $\{B, C\} \subseteq \{A, B\}$? **No!**

Acknowledgments

- Some parts of this presentation was based in publicly available slides and PDFs
  - www.cs.cornell.edu/courses/cs4410/2011su/slides/lecture10.pdf
  - www.microsoft.com/en-us/research/people/madanm/
  - williamstallings.com/OperatingSystems/
  - codex.cs.yale.edu/avi/os-book/OS9/slide-dir/

The END